Understanding the Security Risks of Websites Using Cloud Storage for Direct User File Uploads

Yuanchao Chen, Yuwei Li, Yuliang Lu, Zulie Pan, Yuan Chen, Shouling Ji, *Member, IEEE*, Yu Chen, Yang Li, and Yi Shen

***Abstract*—With the rising demand for website data storage, leveraging cloud storage services for vast user file storage has become prevalent. Nowadays, a new file upload scenario has been introduced, allowing web users to upload files directly to the cloud storage service. This new scenario offers convenience but involves more roles (i.e., web users, web servers, and cloud storage services) and their interactions, bringing new security threats. In this paper, we perform the first systematic security study in this scenario. With in-depth analysis, we identify six new types of vulnerabilities and conduct large-scale real-world measurements on the top 500 Alexa Rank websites. Among these websites, 182 (36.4%) use cloud storage services, illustrating the widespread use of the cloud. Then, we perform a detailed analysis of 28 popular websites that allow user upload. Surprisingly, they all have at least one of the six vulnerabilities. Totally, we discover 79 new vulnerabilities and responsibly report them to the websites. Many popular websites respond positively, including Google, Reddit, and CSDN. We discuss the root causes of these vulnerabilities and propose possible mitigation methods. In summary, our work offers significant value in understanding the security risks of cloud storage services for websites and facilitating future research.**

***Index Terms*—Cloud storage service, web security, upload credential, vulnerability detection.**

I. Introduction

With the ever-increasing popularity of websites, the user base and data volumes of websites increase by a large margin. As a result, storing and managing user-generated massive data become a significant challenge [1]. For example, according to statistical data [2], [3], popular websites like Google and Reddit have hundreds of millions of users. Gmail alone boasts over 1.8 billion users among Google services. Reddit has approximately 430 million monthly active users. Therefore, it is important to address the aforementioned massive users' data storage and management issues. Cloud storage services have emerged as the preferred choice for website data storage and management due to their flexibility, scalability, and pay-as-you-go storage solutions. These features cater to the needs of websites across various sizes and industries [4].

Under the cloud storage service, a new file upload scenario has been introduced, where the files are directly uploaded from web users to the cloud. Generally, there are three critical stages in this new file upload scenario: (1) upload credential requesting and dispatching, (2) file uploading and verification, and (3) callback notification and response. In stage (1), when a user wants to upload a file on a website, the client[1] sends an HTTP request to the web server to ask for an upload credential. Then the web server should check the user's identity and then dispatch the upload credential if passed. In stage (2), the client uses the credential to upload the file directly to the cloud storage service. The cloud storage service should verify the credential's signature information and check whether the uploaded files satisfy the credential's policy requirements. In stage (3), the cloud storage service or the client should send a callback notification to inform the web server of the file upload results.

In contrast to traditional methods where the web server stores user files locally or as an intermediary to transfer files to cloud services [5], [6], this new scenario eliminates the need for the server to manage data storage or the transit of uploaded files. This offers convenience and reduces transmission overhead for web servers. However, it also introduces new attack surfaces, as follows.

Firstly, the process involves authentication interactions among three roles, and each role should be responsible for its own security authentication. For instance, if the web server or cloud storage service fails to provide proper identity authentication for a user, it opens up possibilities for attackers to forge the user's identity.

Secondly, the new scenario introduces a new authorization mechanism—upload credentials. Improper setting of the upload credential or the policy may cause privilege escalation vulnerabilities. For instance, adversaries may exploit these vulnerabilities to steal or modify other users' files in cloud storage services.

Finally, the new scenario involves information synchronization among the three roles. If the web server or cloud

Received 10 May 2024; revised 13 October 2024 and 24 November 2024; accepted 9 February 2025. Date of publication 20 February 2025; date of current version 7 March 2025. This work was supported by the NSFC under Grant 62202484. The associate editor coordinating the review of this article and approving it for publication was Dr. Jianjun Chen. *(Yuanchao Chen and Yuwei Li are co-first authors.) (Corresponding authors: Yuliang Lu; Zulie Pan.)*

Yuanchao Chen, Yuwei Li, Yuliang Lu, Zulie Pan, Yu Chen, Yang Li, and Yi Shen are with the College of Electronic Engineering, National University of Defense Technology, Hefei 230037, China (e-mail: luyuliang@nudt.edu.cn; panzulie17@nudt.edu.cn).

Yuan Chen and Shouling Ji are with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China.

Digital Object Identifier 10.1109/TIFS.2025.3544082

[1]The client refers to the browser side. For the convenience of expression, we regard "user" and "client" as the same meaning in this paper.

storage service does not provide a secure synchronization information verification mechanism, the authenticity of the callback notification information cannot be guaranteed. Then, attackers can forge or tamper with callback notifications to trick the web server.

Previous studies on cloud storage security focus on mobile applications with the misuse of user credentials [7], [8], [9], [10], [11] and the data management issues [12], [13], [14], [15], [16]. However, to the best of our knowledge, no one has studied the security issues of this new scenario. There are two main challenges in analyzing the security risks of this scenario. 1) It is not trivial to understand the complex interaction mechanism between different roles and determine whether it contains vulnerabilities. Many interactions are invisible to a regular web user. For instance, in practice, a user is not able to access the communication process between a website and the cloud storage. 2) It is not trivial to determine whether the concrete implementations or deployments of an interaction mechanism are vulnerable. The complexity arises due to the variety of cloud service vendors and websites, each usually having distinct implementations or deployments.

In this paper, we perform the first systematic study of the security risks of directly uploading files from web users to a cloud storage service. To address the above challenges, we conduct an in-depth analysis of the three critical stages in this scenario based on the public development manuals provided by cloud service vendors. From the perspectives of access control, credential validity, file restrictions, data integrity, data confidentiality, and consistency, we summarize the newly identified vulnerabilities into the following six types: *unrestricted upload credential acquisition (V1)*, *upload credentials validity flaw (V2)*, *unrestricted file types and file size (V3)*, *file overwriting (V4)*, *file stealing (V5)*, and *callback notification spoofing (V6)*. These vulnerabilities can cause serious harm to web users, web servers, and cloud storage services, such as resource-consumption attacks, privacy leakage, sensitive data overwriting, etc.

We manually analyze and pre-collect the keywords and naming model features of various cloud storage services based on the manuals of various cloud storage service providers. Then, we conduct a measurement to evaluate the security risk in the real world by investigating the top 500 popular websites on Alexa. Among them, 182 (36.4%) use cloud storage services, illustrating the widespread use of cloud storage services. Among the 182 websites, we conduct an in-depth security analysis of 28 popular websites that allow user uploading. To our surprise, the experimental results show that all of these 28 websites have at least one of the six types of vulnerabilities. The most common vulnerability is *unrestricted upload credential acquisition (V1)*. During the measurement, we identify a total of 79 new vulnerabilities. We duly report these new vulnerabilities to the security teams of the affected websites and receive positive responses from 10 of them. Moreover, a website (i.e., Academia [17]) has awarded us with bug bounties. To deal with the security risks in this scenario, we discuss the root causes of these vulnerabilities and propose several suggestions for mitigating potential attacks.

In summary, this paper makes the following contributions.

- We conduct the first systematic study on the security risks of directly uploading files from web users to cloud storage. We identify six new types of vulnerabilities and provide a detailed analysis of them.
- Through extensive real-world measurement, we find that all evaluated websites exhibited at least one of the six newly identified vulnerabilities, with a total of 79 new vulnerabilities discovered.
- We report discovered vulnerabilities to the corresponding websites for timely repair. Furthermore, we discuss the root cause of the vulnerabilities and propose feasible approaches to mitigate the potential security risks.

II. Background

In this section, we introduce the details of the scenario for directly uploading files from web users to cloud storage services. Fig. 1 shows the scenario model, which is derived from both the documentation of major cloud providers [18], [19] and manual analysis. The workflow for this scenario contains the following three main stages.

A. Upload Credential Requesting and Dispatching

The process of this stage is illustrated from step ❶ to step ❹. When a web user needs to upload files, the web server will be requested to ask for the upload credentials. Note that different from user credentials, the upload credentials are typically generated temporarily from the user credentials (e.g., IAM [20] and RAM user [21]) according to the predefined security policies. In other words, the upload credentials obtained by the different web users are typically generated from the same cloud service account. From the perspective of the cloud storage service, users are granted identical permissions. The policies may include the limitations of the allowed file types, size, storage paths, credential expiration times, etc. The web server checks whether the user is logged in, and has the upload privileges. If the verifications are passed, the web server will dispatch the upload credentials to the user in two common ways. (i) The upload credential is signed according to the pre-shared user credential when the website developer configures the cloud storage service. In this case, the web server does not need to interact with the cloud storage service. (ii) The web service calls the cloud storage service temporary key generation API according to the information of the user-uploaded file to apply for a temporary upload credential. The credential encodes the authorization policy configured by the web server (e.g., Alibaba Cloud OSS Temporary Identity Authorization STS service) [22]. During this process, the cloud storage service needs to authenticate the web server, usually through the cloud storage service's access key (such as the 'AccessKey ID' and 'AccessKey Secret' of the IAM user and RAM user). After obtaining the upload credential, the web server returns it to the user. During this process, the security of the user's identity, permissions, and uploaded files is of paramount importance.

B. File Uploading and Verification

The process of this stage is shown in step ❺. With the received upload credential, the client will upload the file to the

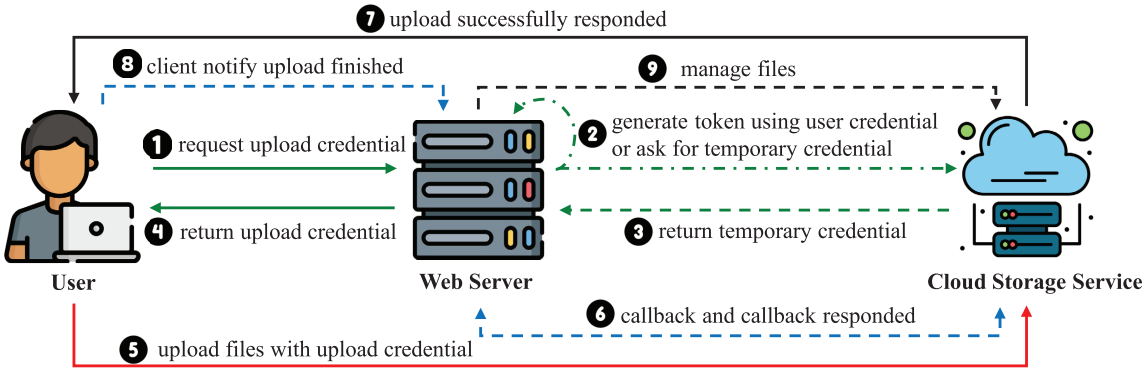


Fig. 1. The scenario model of directly uploading files from a web user to the cloud storage service. The solid lines represent mandatory steps in this process, the dashed lines represent optional steps, and the dash-dot indicates a choice between two steps. The green, red, and black lines represent the steps of the three different stages.

cloud storage service. In this process, since the website user is not a direct user of the cloud storage service, the cloud storage service usually does not authenticate the website user. The cloud storage service should perform security checks on the uploaded file and the upload credential. For the uploaded file, the security checks could include 1) whether it can overwrite the existing file; 2) the file's properties, e.g., whether the size or type of the file satisfies the requirements of the upload policy. For the upload credential, the cloud storage service should check its validity. For instance, if the user's upload credential is expired, the upload request should not be allowed [23]. Overall, the cloud storage service does not pay attention to whether the user initiating the upload request is a legitimate user of the website, it only verifies the legitimacy of the request through the upload credential contained in the request.

C. Callback Notification and Response

The process of this stage is shown in steps ❻ to ❽. In most cases, after a user uploads a file, the web server needs to know the upload results, such as which files the user has uploaded, the name, the size of the uploaded file, etc. Therefore, cloud storage services usually provide callback notification APIs (Application Programming Interfaces) for sending the upload results to the web server. Specifically, there are two main ways for sending the callback notification to the web server. One is the cloud storage service sends the notification directly to the web server (step ❻). The other is the cloud storage service sends the notification to the client, and then the client transfers it to the web server (step ❼ and ❽). In this process, the web server should verify the signature of the callback notification to prevent attackers from forging and tampering with the callback notification.

In addition, the cloud storage services may provide some APIs for web servers to manage objects in the cloud (step ❾). For instance, when a verification error (e.g., insufficient user upload space quota) occurs, the web server can actively call the related APIs to delete the uploaded file [24].

III. Threat Model and Vulnerability Analysis

In this section, we introduce the threat model and the six types of vulnerabilities in this scenario.

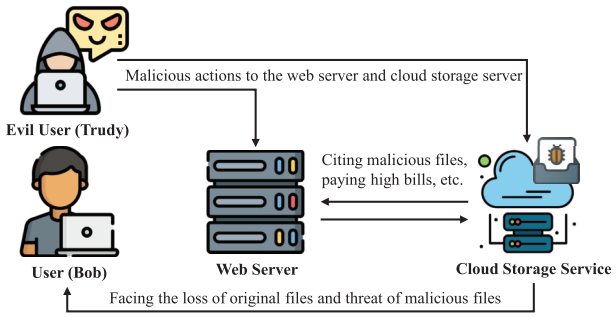


Fig. 2. The threat model of directly uploading files from web users to the cloud storage service.

A. Threat Model

Fig. 2 illustrates the threat model in the scenario of directly uploading files from web users to the cloud storage service. There are four roles in this threat model: the web server, the benign web user (e.g., Bob), the evil web user (e.g., Trudy), and the cloud storage service. The website runs on a web server and uses cloud storage services to save files uploaded by web users. Among the roles in this threat model, only Trudy is malicious. The malicious user Trudy has only the same privileges as a regular user (e.g., Bob), who is also a regular website user. The goal of Trudy is to compromise the web server or the cloud storage service, threatening the security of other benign users like Bob. Trudy can initiate a normal file upload operation on the website, such as uploading an avatar. During this process, Trudy can intercept the HTTP request for upload credentials, the upload credentials themselves, the HTTP request for the file upload, and the callback data packet. By analyzing these packets, Trudy can tamper with specific fields within them to execute various attack behaviors. For instance, if the website has a file overwriting vulnerability, Trudy can modify the file storage path and file name in the upload credentials to match those of user Bob, thereby launching a file overwrite attack targeting that user. This threat model is applicable to cloud hosting and website-cloud integration scenarios, which also involve interactions between these three roles.

B. Vulnerability Analysis

By systematically analyzing the interactions in the cloud storage scenario, we identify the following six new types of vulnerabilities.

1) Unrestricted Upload Credential Acquisition (V1): As mentioned above, if a web user wants to upload a file, he needs to send an upload credential request to the web server. The web server typically performs security checks to determine whether to dispatch the credentials. Nevertheless, improper security checks may cause the following issues. (1) The web server provides the upload credentials without verifying the user's identity. This issue allows users to obtain a large number of upload credentials from the web server by faking an upload credentials request without the current website account. (2) The web server does not check whether the user already has an upload credential and whether his upload credential has expired when it receives an upload credential request. This allows users to send a large number of upload credentials requests to the web service using their account in a short period, thus obtaining a large number of upload credentials. Notably, some websites support anonymous users to upload files to cloud storage services. In such scenarios, the management of upload credentials and the limitation of the frequency of requested file uploads are also crucial.

Therefore, Trudy can easily carry out abuse attacks with the above issues [25]. Suppose the web server does not verify the identity of Trudy. In that case, she can obtain a large number of upload credentials. Then, Trudy is able to use these credentials to upload a large number of files simultaneously, taking up the upload and storage resources of the cloud storage service. In addition, the cloud storage services are billed by storage space, number of requests, or number of API calls (e.g., file upload API) [26], [27], [28]. This malicious behavior may cause the web server to bear high bills for the cloud storage service [29].

2) Upload Credentials Validity Flaw (V2): In this file upload scenario, the validity of the upload credentials is essential for security. The validity for the upload credential includes (1) the validity period and (2) the number of times it can be used. The cloud storage service provides website developers with APIs to configure the validity of the uploaded credentials. The file upload process for cloud storage services is usually fast, automatic, and almost entirely transparent to users. After the user completes a file upload, even if the old credential is still valid, the server will issue a new credential to support the new upload process when the user initiates a file upload request again. Therefore, the validity period of the upload credential generally only needs to cover a single upload process. However, there is a common problem in the upload credential configuration of real websites: many web developers configure the upload credential policy with an excessively long validity period and no restriction on the number of times (e.g., fiverr.com, notion.com, etc.). In that case, the resulting upload credentials may be challenging to manage and control, and attackers can exploit this issue to abuse upload credentials.

For instance, pinterest.com sets the validity period of its upload credentials to two days, with no limit on the number of times these credentials can be used. Thus, Trudy can exploit this flaw to upload a large number of files within the validity period of the upload credential, which will occupy a large amount of storage space resources of the cloud storage service, causing the website to bear high bills for the cloud storage service. In addition, this flaw may result in the website losing control over the usage of the upload credentials. Typically, the websites only allow their users to store files in the associated cloud storage service. In that case, Trudy may still be able to upload files to the cloud storage service or access its data even after her website account permissions have expired.

3) Unrestricted File Types or File Sizes (V3): When website developers do not restrict the type or size of files that can be uploaded in the upload credentials, this will open the door for attackers. They could potentially upload malicious files that jeopardize the safety of web users, or they may upload enormous files that exhaust storage space, thereby affecting the normal operation of cloud storage services. For instance, this issue provides conditions for the security threats and attack methods mentioned by Hong et al. [30] and Lv Y et al. [31].

If the types of files are not restricted, Trudy can upload any file using this credential. Upon Trudy's request to upload a 1 MB picture, the web server dispatches her an upload credential after determining that the picture file type and size meet the security requirements. However, as the file type is not signed in the upload credential, Trudy could potentially misuse this privilege. Instead of uploading the agreed picture, she could use the credential to upload a malicious file of the same size, 1 MB. For instance, websites typically implement a Content Security Policy (CSP) [32], [33], which is a set of rules defined by administrators to control the sources from which a webpage can load and execute content. However, tens of millions of websites include cloud storage root domains in their CSP allowed list [31]. This situation provides the possibility for attackers to exploit this vulnerability to launch web front-end attacks, such as Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and Clickjacking [34], [35], [36], [37], [38]. Trudy can upload HTML files that contain malicious JavaScript code and then lure users into accessing these files, stealing user information.

If the size of allowing upload files is not restricted, Trudy can upload any file size to the cloud storage service. Just like the file type is not limited, Trudy can use this flaw to bypass the file size limit of the web front end. Trudy can upload a huge file to maliciously consume the cloud storage service's storage resources, affecting other users' use and causing the web server to incur high bills. On the other hand, Trudy can combine it with V1 and V2 to perform resource consumption attacks on cloud storage services.

4) File Overwriting (V4): There are two main conditions of this vulnerability: (1) the file storage path in the upload credential can be tampered with by users, and (2) the existing files can be overwritten in the cloud storage service. In the file uploading and verification stage, since the website user is not a direct user of the cloud storage service, the cloud storage service does not authenticate the website user. Instead, it verifies whether the file upload request includes the upload credential and meets the requirements through the upload credential. When the website developers configure the cloud

TABLE I
CLOUD STORAGE SERVICE FILE OVERRIDE CONFIGURATION INFORMATION

Service	Default Overwrite	Support Versioning	Other Control Methods
Amazon S3	✓	✓	✗
Alibaba OSS	✓	✓	x-oss-forbid-overwrite
Tencent COS	✓	✓	✗
Azure Cloud	✓	✓	–overwrite
Baidu BOS	✓	✓	x-bce-forbid-overwrite

storage service, they can set the file overwriting-related field in the upload credential. This field indicates whether the user can overwrite the original file when uploading a file whose name already exists in a cloud storage service. As shown in Table I, the well-known cloud storage services such as Amazon S3 [39], Alibaba Cloud OSS [22], Azure cloud storage [40], Tencent COS [41], and Baidu BOS [42] allow overwrite files with the same name by default. These cloud storage services can prevent files with the same name from being overwritten by turning on versioning. Alibaba Cloud OSS, Baidu BOS, and Azure cloud storage can control whether to overwrite the file with the same name in the Bucket by changing the value of the corresponding field in the HTTP request without enabling versioning. For example, setting `x-oss-forbid-overwrite` to true will not overwrite files with the same name. However, website developers who configure cloud storage services usually use the default configuration because of forget or to reduce space overhead.

If Trudy has an upload credential that does not have signature restrictions on the file name and storage path, she can perform a file overwriting attack on Bob using this configuration issue. Trudy can change the storage path and file name of the uploaded file in the upload credential to be consistent with Bob's uploaded file in the cloud storage. Suppose Trudy gets the information about the file stored in the Bucket, which Bob uploads (we call it $file_{Bob}$). In that case, she can upload a file with the same name and storage path as $file_{Bob}$, which will overwrite $file_{Bob}$ stored in the Bucket, which will directly affect Bob's security. This malicious action results in losing Bob's original files uploaded to the cloud storage service. What's more serious is that Trudy uses the file containing the attack payload to overwrite $file_{Bob}$. For example, the document file includes the attack payload of CVE-2017-11882. If Bob downloads $file_{Bob}$ to his personal computer, he may face a security threat from Trudy's attack. Even worse, it is highly possible for Trudy to gain control of Bob's computer.

5) File Stealing (V5): A cloud storage service is usually configured with calling interface operations related to Buckets and Objects. These interfaces offer convenience to website developers managing the Bucket and users uploading and downloading objects. For example, Alibaba Cloud OSS provides users with GetBucket and GetObject interfaces. Amazon S3 provides users with ListObject, and GetObject interfaces, which are used to list all objects in the Bucket and get object information [43], [44]. When the users have the corresponding permission, they can use the above interface to get the list of all files stored in the Bucket, the information of the specified file, or even download the file. The cloud storage services are also configured with an access control list (Bucket ACL) [45] for Bucket read and write permissions. The cloud storage service determines the permission access control of website developers and web users through Bucket ACL, which mainly includes reading class, writing class, and global type permissions. When the cloud storage service receives a request for a resource, it first checks the corresponding ACL to verify whether the requester has the necessary access privileges. If the interface permissions are not set properly, the users' private files in the Bucket will be at risk of being stolen by attackers.

In the process of Trudy uploading files to the cloud storage service, she can perform a file theft attack in the following two situations. (1) The upload credential dispatched to her has the access rights to call the interface of listing all files in the Bucket and the permission to access all files in the current Bucket. (2) Website developers set the Bucket ACL to `public-read` type permissions when configuring cloud storage services, and the upload credentials dispatched to her have the permission to call the interface to list Bucket. In these situations, Trudy can get the information of all files in the current Bucket and steal arbitrary files. This issue will bring a great security threat to user privacy security.

6) Callback Notification Spoofing (V6): The cloud storage service should set callback notifications to inform the web server about the upload information, such as whether the upload operation has been completed. However, there are some deviations in the callback notification settings of various cloud storage service vendors. Here are two typical issues. (1) The cloud storage service does not provide a callback notification interface. After completing the file upload, the client (website front end) sends a callback notification to the web server. (2) The callback notification provided by the cloud storage service does not have a signature, or the web server may perform relevant processing operations without verifying the authenticity of the callback notification. Trudy can use these issues to generate fake callback notifications or maliciously tamper with callback notifications to initiate spoofing attacks.

Listing 1 Example of callback notification spoofing.

```
router.post('/callback/wcs', (req, res) => {
    ResponseUtil.tryRun(req, res, async () => {
        let data = await Rpcs.cloudStoreService.up-
        loadFile(req.body.callbackBody)
        let names = data.originalFilename.split('
        |@qzy_inner@|')
        let parent = names[0]
        let name = names[1]
        let path = names[2]
        let userId = data.uploadUser
        if (IceUtil.iceLong2Number(userId) < 1) {
            data.originalFilename = names[1]
            return data
        }
        return await copyFileDataToUserSpace(userId,
        parent, path, name, data)
    })
})
```

a) Generating Fake Callback Notifications: Trudy can generate fake callback notifications to deceive the web server. For example, Trudy uses fake notifications to inject advertisement files into other users' space in a network disk application

TABLE II
KEYWORDS AND NAMING PATTERNS CHARACTERISTICS OF SOME CLOUD STORAGE SERVICES

Service	Front-end source code	File upload Packet	HTTP Response Header	Domain of the file storage URL
Amazon S3	aws-sdk-<version>.min.js, aws-sdk-<version>.js, AWS.S3, s3.upload s3:PutObject, ...	X-Amz-Algorithm, X-Amz-Credential, X-Amz-Date, X-Amz-Expires, X-Amz-Signature, x-amz-content-sha256. ...	Server: AmazonS3, x-amz-id-2, x-amz-bucket-region, x-amz-expiration, x-amz-request-id, Etag, ...	<bucket-name>.s3.<region>.amazonaws.com.cn, <bucket-name>.s3-website.<region>.amazonaws.com, <bucket-name>.s3-website-<region>.amazonaws.com, <bucket-name>.s3.<region>.amazonaws.com.<area-suffix>, ...
Alibaba OSS	aliyun-oss-sdk-<version>.min.js, aliyun-oss-sdk-<version>.js, require('ali-oss'), oss.upload, ...	authorization: OSS*, x-oss-security-token, x-oss-meta*, OSSAccessKeyId, access_[$key\|\|token\|\|id$], ...	Server: AliyunOSS, x-oss-request-id, x-oss-server-time, x-oss-object-type, Etag, ...	<bucket-name>.oss-<area-suffix>-<region>.aliyuncs.com, <bucket-name>.oss-<area-suffix>-<region>-internal.aliyuncs.com, <bucket-name>.oss-accelerate.aliyuncs.com, <bucket-name>.oss-accelerate-overseas.aliyuncs.com
Tencent COS	cos-nodejs-sdk-<version>, qcloud-cos-sts, cos.putObject, ...	q-sign-algorithm, q-sign-time, q-signature, q-ak, x-cos-security-token, x-cos-meta-*, ...	Server: tencent-cos x-cos-hash-crc64ecma, x-cos-request-id, x-cos-server-side-encryption, Etag, ...	<bucket-name>-<appid>.cos.<region>.myqcloud.com, <bucket-name>-<appid>.cos.accelerate.myqcloud.com, <bucket-name>-<appid>.cos-internal.accelerate.tencentcos.cn, ...
Azure Cloud	@azure/storage-blob, BlobServiceClient.uploadBlob, BlockBlobClient.upload, ...	x-ms-blob-content-type, x-ms-meta-*, x-ms-expiry-time, x-ms-encryption-algorithm, x-ms-encryption-key, sv=,sp=,sp=,se=,sig=, ...	Server: Windows-Azure-Blob, x-ms-request-id, x-ms-version, x-ms-version-id x-ms-content-crc64, Etag, ...	<storage-account>.blob.core.windows.net, <storage-account>.z[00-99].blob.storage.azure.net, <storage-account>.web.core.windows.net, <storage-account>.file.core.windows.net, ...
Google Cloud Storage	generateV4UploadSignedUrl.js, generateV4SignedPolicy.js @google-cloud/storage, gcs.upload() ...	generateV4UploadSignedUrl.js, generateV4SignedPolicy.js @google-cloud/storage, gcs.upload() ...	Server:UploadServer x-goog-Upload-Url, x-guploader-Uploadid, x-goog-Upload-Status, Etag, ...	<bucket-name>.storage.googleapis.com, *.storage.googleapis.com/<bucket-name>/

or make the server ignore real notifications. As shown in Listing 1 is the code for an open-source network disk service to process the callback notification of the cloud storage service [46]. Line 4 shows that the network disk data comes from the callback notification the cloud storage service sends after the user uploads successfully. However, the network disk doesn't carry out signature verification on the callback notification, which leads to malicious users (e.g., Trudy) can generate fake callback notifications. Attacks can inject malicious files into other users' network disk space, such as files containing advertisements, gambling, or explicit content.

b) Malicious Tampering Callback Notification: Trudy can maliciously tamper with the callback notification and cause security threats to the web server. The web server performs related operations based on the callback notification information, such as logging file information into the database or transferring specific types of files into the local system. Trudy can maliciously tamper with the callback notification and send malicious data, incorrect data, and even the payload of specific vulnerability exploits, such as Server-Side Request Forgery (SSRF), to the web server, which poses a security threat to the web server.

IV. EXPERIMENTS

To understand whether and how popular websites in the real world are affected by cloud storage service vulnerabilities, we first conduct a measurement by investigating the cloud storage service usage of the top 500 Alexa Rank websites[2]. Then, we perform a detailed analysis of 28 popular websites that use cloud storage services.

A. Usage of Cloud Storage Services

1) Identification Method: We peruse the manuals of various cloud storage service providers and identify cloud storage services from the following aspects according to the manuals. The analysis targets mainly include the website's front-end source code, the interactive packets used to upload files, the HTTP Response Header returned when requesting the file storage address, and the domain of the file storage URL. We manually analyzed and pre-collected the keywords and naming model features of various cloud storage services, some of which are shown in Table II.

For the website that provides the function of directly uploading files to the cloud storage service, the JavaScript SDK provided by the relevant cloud storage service will be referenced on the front-end source code. These SDKs can be identified through their file names or JavaScript script content. For example, the file name of the JavaScript SDK provided by Alibaba Cloud OSS contains the string "aliyun-oss-sdk-[version]". In addition, during the file uploading process, we analyze whether the web server or cloud storage service will dispatch an upload credential to the user. The type of cloud storage service can then be determined based on the field information within the upload credential. If no upload credentials are provided, we analyze the domain name in

[2]Taken from Alexa Rank and the time of this data is: November 8, 2021, at 9:00 am.

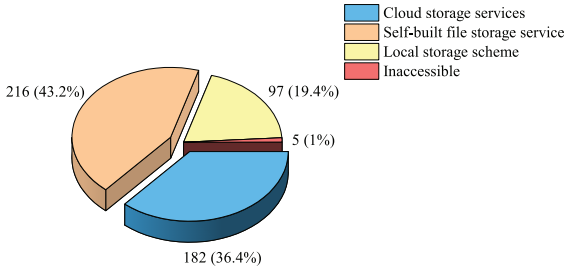


Fig. 3. The usage of cloud storage services.

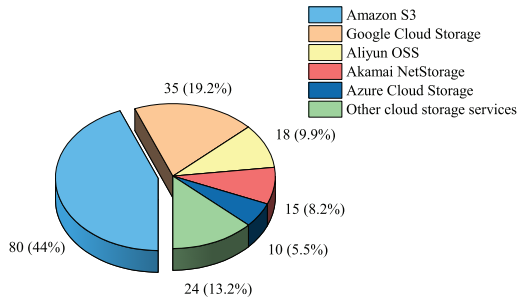


Fig. 4. The usage of public cloud storage services.

the file URL, such as "*.amazonaws.com" for Amazon S3. Additionally, we analyze the HTTP Response Header returned by requesting the file URL address. The response header of the cloud storage service usually contains its unique fields or field values. For example, as shown in Table II, the "server value" of Amazon S3 is usually "AmazonS3", and "x-amz-id-2", "x-amz-bucket-region", or "x-amz-request-id" fields appear in the HTTP Response Header.

For websites that do not offer a file upload function, we gather the URLs of images, videos, and document files. We filter out the file URLs that are different from the second-level domain name of the target website and only access those with the same second-level domain name. We determine whether the website uses a cloud storage service and which cloud storage service is used through the HTTP Response Header, the domain of the file storage URL, and the response characteristics of accessing the root path of the file URL.

Without the fingerprint above features, we cannot determine whether or not the target websites utilize a cloud storage service, nor can we identify which specific cloud storage service they use. For these websites, if the file URLs have the same second-level domain name as the target website but a different third-level domain name, we conclude that the website utilizes a self-built file storage server. Conversely, if the domain names of the file URLs are the same as the target website, we conclude that the website uses a local storage scheme.

To ensure the accuracy of cloud storage service usage measurements, we invited 3 researchers to analyze the top 500 Alexa Rank websites manually. It took each researcher an average of 5 days to complete the measurements for all websites. We compared the measurement results obtained by the three researchers and manually re-verified the sections of the websites that showed discrepancies. Finally, we collected the usage data of cloud storage services across the top 500 popular websites.

2) Usage: The statistical results of the cloud storage service usage of the 500 websites are shown in Fig. 3. Among the 500 websites analyzed, 182 were found to utilize cloud storage services for storing user data such as images, videos, or files, representing a utilization rate of 36.4%. Another 216 websites use self-built file storage services. 97 websites use traditional local file storage solutions. For the remaining five websites, we can not confirm or access their utilization of cloud storage services due to their temporary or permanent unavailability.

For the 182 websites that we identify as using cloud storage services, we perform further analysis to determine which specific cloud storage service they use. Our analysis results are shown in Fig. 4. The analysis results indicate that the top cloud storage services most often utilized by the Alexa Rank top 500 websites include Amazon S3, Google Cloud Storage, Akamai Storage, Alibaba Cloud OSS, and Microsoft Azure Storage.

B. Experimental Target Selection

To verify how widespread popular websites' cloud storage service vulnerabilities are, we select experimental targets from the 182 websites that use cloud storage services. We focus on those websites that support user registration, and users can upload files to cloud storage services. We exclude those sites for which we are unable to register accounts.

We analyze these 182 websites, and there are 28 websites that meet the above conditions. Most of the websites are platforms for social media, file processing, and data sharing, where the user can upload files such as pictures, videos, documents, etc. We conduct a detailed analysis of the process of directly uploading files from users to cloud storage services on these websites. Table III shows these websites' ranking, i.e., popularity.

C. Experiment Methodology

The experimental methodology is divided into two parts: static analysis and dynamic probing. In static analysis, we examine upload credentials for security risks like expiration time, file restrictions, and storage path control. In dynamic probing, we modify upload requests to test credential issuance, usage limits, file overwriting risks, access control, and callback security.

Specifically, we create $user_{Trudy}$ and $user_{Bob}$ in each website to simulate the attacker and victim and complete an effective file upload operation on the target website, such as uploading an avatar. We can extract the HTTP request for obtaining the upload credentials, the upload credentials themselves, the HTTP request for file uploading, and the callback packet from $user_{Trudy}$'s file upload process [47]. It should be emphasized that we do not pose any security threat to other users while verifying the existence of the cloud storage service vulnerability.

TABLE III
THE INFORMATION OF THE 28 TARGET WEBSITES AND THE VULNERABILITY ANALYSIS RESULTS

#	Alexa Rank	Domain	Storage Provider	V1		V2		V3			V4	V5	V6
				Unrstr	Noauth	Expiration	Use Limit	Type	Size	T & S			
1	1	google.com	Google	✓		None	✓	✓					
2	2	youtube.com	Google	✓		None	✓	✓					
3	4	sohu.com	Kuaizhan		✓	None	✓						
4	18	live.com	Azure	✓		5D	✓		✓				
5	19	reddit.com	Amazon	✓		1m							
6	27	csdn.net	Alibaba	✓		10m				✓	✓	✓	✓
7	30	twitch.tv	Amazon	✓		15m				✓			
8	56	fiverr.com	Amazon	✓		9h		✓			✓		
9	70	canva.com	Amazon	✓		1h							
10	99	udemy.com	Amazon	✓		User Define			✓		✓		
11	104	instructure.com	Amazon	✓		10m				✓			
12	111	pinterest.com	Amazon	✓		2D		✓			✓		
13	162	wetransfer.com	Amazon	✓		2h		✓					
14	163	archive.org	Amazon			None				✓			
15	167	notion.so	Amazon	✓		1D				✓			
16	189	vimeo.com	Google	✓		1m			✓				✓
17	192	smallpdf.com	Amazon		✓	15m				✓	✓		
18	203	padlet.com	Google	✓		30m		✓					
19	224	blogger.com	Google	✓		None							
20	274	tudou.com	Alibaba	✓		1h				✓			
21	288	discord.com	Google			None				✓			
22	298	realtor.com	Amazon	✓		5m							
23	303	dailymotion.com	Amazon	✓		10h	✓			✓			
24	326	zhihu.com	Alibaba	✓		9h				✓	✓		
25	342	slideshare.net	Amazon	✓		12h				✓	✓		
26	346	patreon.com	Amazon	✓		24h				✓			
27	424	academia.edu	Amazon	✓		30m			✓				
28	474	onlyfans.com	Amazon			None				✓			

Unrstr indicates that the user can get a large number of upload credentials without restriction, and Noauth indicates that the web server dispatches the upload credentials without detecting the user's identity. Expiration indicates the validity period of the uploaded credential, and Use Limit represents whether to limit the number of times the credential can be used.

Listing 2 Plaintext data after Policy decoding.

```
{
  "expiration": "2022-05-25T17:31:29.580Z",
  "conditions": [
    {"acl": "public-read"},
    {"bucket": "image-web-upload"},
    {"Content-Type": "image/jpeg"},
    {"success_action_status": "200"},
    {"starts-with": $key},
    {"x-amz-meta-qqfilename": "source.jpg"},
    ["content-length-range", "1", "10000000"]
  ]
}
```

1) Static Analysis: We statically analyze the policy or signature information in the upload credentials. For example, for the `HTTP POST requests` method provided by Amazon S3 in Fig. 5, the upload credential dispatched by this method contains the "policy" which is usually base64-encoded data, the signature "signature" of the policy, and the "AWSAccessKeyId" field, etc. Listing 2 shows the plaintext data after policy decoding in the sample HTTP POST credentials. The policy usually includes information such as credential expiration time field `expiration`, the file ACL field `acl`, file storage path and name field `starts-with`, allowed uploaded file type field `content-Type` and file size field `content-length-range` [48], etc. We perform security analysis on each field of policy information. Specifically, we get the validity period from the credential or calculate the validity period of the upload credential by the difference between the expiration time in the upload credential and the time in the HTTP Response. Then, we check whether the policy includes fields of file type and file size. If there is no relevant field, we will determine whether the current upload credential can be used to upload files of other types or sizes during dynamic probing. Finally, We judge whether the policy contains the file storage path and name information and whether they are under control. For example, if the "starts-with" field has the variable $key, it means that the user can customize the storage path and name of the file.

2) Dynamic Probing: For the V1, we repeat $user_{Trudy}$'s original HTTP request and the HTTP request without authentication information for requesting the upload credential. By comparing new and existing credentials, we determine if a

Fig. 5. Amazon S3 HTTP POST credential example.

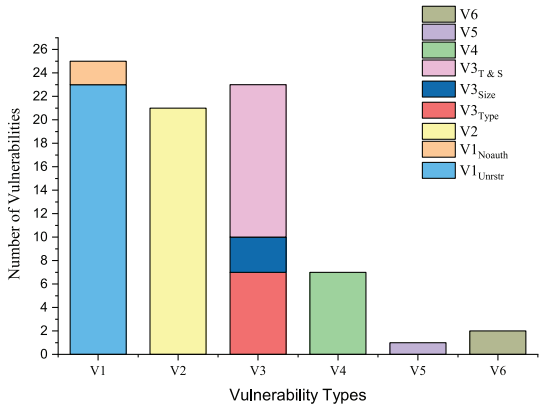


Fig. 6. The number of vulnerabilities of each type.

web user can obtain unlimited credentials or if the website issues them without identity verification. Then, we test if the upload credential limits usage by attempting multiple uploads with a single credential, checking for successful uploads each time. For V3, we adjust the upload file type or size to test if $user_{Trudy}$ can upload any file type or size using the same upload credential. If the file upload is successful, we access the file multiple times. This approach prevents false positives due to some opaque processing logic of the website after the file is uploaded.

We validate V4 again during the dynamic probing. In V4, we change the file storage path and name in the policy to match $file_{Bob}$, using $user_{Trudy}$'s identity. If the file upload is successful, we download the $file_{BOb}$ to determine whether the $file_{BOb}$ is overwritten by $user_{Trudy}$. For V5, we use $user_{Trudy}$'s upload credentials to access the Bucket, where Bob's file is stored. We try to call interfaces like GetBucket and GetObject to judge if $user_{Trudy}$ can access or download any file in the Bucket. Finally, we check the possibility of forging or tampering with callback notifications. If the website has set callback notifications, we modify their content, including changing URLs, to check for vulnerabilities in notification spoofing based on the response and DNS resolution record data.

D. Cloud Storage Service Vulnerabilities in Popular Websites

We comprehensively analyze the process of directly uploading files from web users to cloud storage services used by 28 websites. The experimental results are shown in Table III. The Storage Provider column indicates the primary cloud storage service used by the website. Columns V1 to V6 indicate the cloud storage service vulnerabilities of the website. The statistics of V1 to V6 are shown in Fig. 6.

1) Summary of the Detected Vulnerabilities: From Table III and Fig. 6, we can get the following observations. First, all of these 28 websites have at least one of the six types of vulnerabilities, which demonstrate that this scenario is facing severe security threats. During the experiments, we identify 79 new vulnerabilities and have report them to the developers. Second, the most common vulnerability is V1. Only 3 of 28 websites do not have V1. Third, the number of websites that have V5 and V6 is the lowest. In the following, we present a detailed analysis of the detected six types of vulnerabilities in the 28 websites.

a) Unrestricted Upload Credential Acquisition (V1): We find that 25 websites face the security threat of V1. Among them, 23 websites do not check whether the user has already got an upload credential and whether the user's upload credential has expired. Two websites provide the upload credentials without verifying the user's identity. SmallPDF supports anonymous users uploading files to cloud storage services. However, anonymous users can also get a large number of upload credentials to pose a threat to websites and cloud storage services.

b) Upload Credentials Validity Flaw (V2): We conduct an objective discussion on the expiration and usage limit of the upload credential configured by each website. As mentioned, the upload credential only needs to support completing the file upload process once. When this process is completed, the credential should immediately become invalid. However, among the 28 websites evaluated, there are only four websites that limit the number of times uploaded credentials can be used. 18 websites have set a fixed validity period for their upload credentials and allow unlimited usage within this specific timeframe. The validity periods of these upload credentials vary, with the shortest being as brief as 1 minute, while the longest extends to two days. We experimentally test the time required to complete a compliant file upload on these websites (e,g, uploading an avatar or a document). It takes approximately 1.2 seconds to upload a file of 5MB on average. There is one website that can allow users to customize the expiration time of the upload credential. Even more shocking are four websites that neither set a validity period for their upload credentials nor implement a usage limit.

c) Unrestricted File Types and File Size (V3): For V3, among the 23 websites that face the threat of V3, 13 websites have no restrictions on the file size and type of the uploaded files, and the rest have restrictions on one of the size or type of files in their upload credentials.

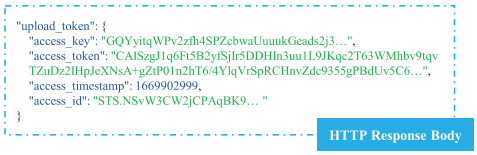
d) File Overwriting (V4): There are seven websites that face V4 threats, and attackers can easily launch file overwrite attacks on target users on these websites. Among them, during the credential application phase of Udemy, attackers can request and generate upload credentials of any storage location to overwrite any files in Bucket. The other six websites are because the file storage path and file name are not signed restrictions in the credentials.

e) File Stealing (V5): Only one website (i.e., csdn.net) has the V5 vulnerability. This is because it does not implement

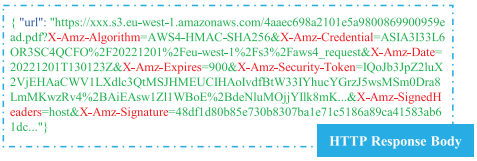
(a) An upload credential example for Google cloud storage

(b) An upload credential example dispatched to Live by Onedrive

Fig. 7. Two upload credentials example for google cloud storage and onedrive.



(a) An upload credential example for Alibaba Cloud OSS STS service



(b) Amazon S3 query parameters method upload credential example

Fig. 8. Two upload credentials example for alibaba cloud OSS and amazon S3.

the principle of least privilege for the upload credentials, resulting in the upload credentials having the permission to list all files in the bucket and download any file.

f) Callback Notification Spoofing (V6): In the callback notification and response stage, it can be seen from Table III that two websites (i.e., csdn.net, vimeo.com) face the threat of V6. The attacker can tamper with some information in the callback notifications of these two websites to affect the normal operation of the website.

E. Cast Studies

To better understand the security risks, we perform several case studies on some websites to explain the vulnerabilities and how attackers can exploit them as follows.

1) Google and YouTube: Google and YouTube both use the Google Cloud Storage service to store files uploaded by users. When a user sends an upload credential request, the Google and YouTube servers dispatch the upload credential `upload_id` information and the URL for uploading files to the user as shown in Fig. 7(a). The `upload_id` and the URL for uploading files are in the `X-Guploader-Uploadid`, and `X-Goog-Upload-Url` field in the HTTP Response Header [49]. Users can use `upload_id` to upload files to the cloud storage service. Google and YouTube have limited the number of times the upload credentials can be used. After the user has only performed one file upload operation using the upload credential, the upload credential becomes invalid. This is a security measure to prevent users from abusing it by repeatedly uploading. However, Google and YouTube do not verify whether the user has received the upload credentials and do not limit the number of upload credentials that the user can get. Therefore, the attacker can obtain a large number of `upload_id` information by replaying the HTTP request, resulting in the above security measures being bypassed. Additionally, the `upload_id` does not have a signature limit on the file size of uploaded files so an attacker can upload files of any size. It is important to emphasize that the upload credentials provided by the website correspond to the cloud resources opened by the website and consume the website's bill. Therefore, an attacker can generate a large number of `upload_ids` and simultaneously initiate uploads, maliciously consuming the website's bandwidth and cloud storage resources. This can lead to significant strain on the website's infrastructure and increased operational costs.

2) Live: Live uses Microsoft's cloud storage service, Azure Storage. The users can upload files to Onedrive through the attachment upload functionality when sending emails. When a user sends an attachment upload request to Live, the user can get an upload credential dispatched by Live through the `uploadUrl` field with a validity period of five days as shown in Fig. 7(b). After a detailed analysis of Live, we find that although the upload credential dispatched by Live is five days valid, it can no longer be used after it has been used once. Therefore, we judged that Live is not affected by V2. After an attacker logs in to Live, he can get a large number of upload credentials by replaying the HTTP request. The attacker can get the upload credential signed by any file type by changing the file extension name in the HTTP request, and the upload credential does not limit the file size. The cloud storage service vulnerabilities also threaten other Microsoft websites that can upload files to Onedrive.

3) CSDN and ZhiHu: Both CSDN and ZhiHu, utilizing Alibaba Cloud OSS, provide temporary upload credentials to users via OSS's STS service. The upload credential example is shown in Fig. 8 (a). The web server, upon such requests, configures the user's access policy and credential validity via the `AssumeRole` request to Alibaba Cloud OSS. Then, these credentials are relayed to users for file uploads using OSS API.

We find that by changing the hash[3] in the HTTP request, users can get multiple upload credentials on CSDN and ZhiHu. These credentials, lacking limitations on file type and size, remain valid for excessively long periods (10 minutes for CSDN and 9 hours for ZhiHu). Additionally, the absence of the `x-oss-forbid-overwrite` setting and version control on both platforms allows users to overwrite existing files in the Bucket. This vulnerability could enable attackers to replace images in published articles with content contain-

[3]A field in the HTTP request packet, the value is the MD5 of the file, and the website uses it for deduplication.

ing illegal or harmful information, such as gambling or pornographic links, leading to the dissemination of malicious content.

Moreover, CSDN does not impose strict permission restrictions on the upload credentials when requesting `AssumeRole`. As a result, an attacker can access and steal any file in the Bucket, which contains 83.63TB of user data, by calling GetBucket and GetObject interfaces in Alibaba Cloud OSS. CSDN's callback notifications, used to save files to "img-blog.csdnimg.cn", have a critical flaw as they lack signature verification. This oversight permits attackers to manipulate callback data, enabling them to overwrite images in other users' articles with harmful content, including advertising, gambling, or pornography. This vulnerability also opens the door for Server-Side Request Forgery (SSRF) attacks through tampered callback data.

4) Udemy and Smallpdf: Udemy and Smallpdf use Amazon S3, a cloud storage service provided by Amazon. They use the methods of `HTTP POST Requests` and `Query Parameters` provided by S3 for authentication, and the signature identifier is `X-Amz-Signature` field [50], [51]. An example of the upload credential generated by the `Query Parameters` method is shown in Fig. 8(b). Udemy generates HTTP POST credential information based on the file information uploaded by the user. This method is widely used by most websites, which use Amazon S3 cloud storage service to generate upload credentials. Due to the Udemy administrator's misconfiguration, the HTTP request to send file information to Amazon S3 is sent by the client, which results in the user being able to change or fake the upload credential request. For example, the user can customize the storage location of the file and the file name in the Bucket, the file size, file type, and the expiration time of the upload credential, etc. The user can send the customized HTTP request to Amazon S3 to get the uploaded credentials. This is undoubtedly a severe security issue. An attacker can exploit this issue to get the signing upload credential of any file, and he/she can overwrite any file in the Bucket.

Smallpdf provides users with a file conversion service, which requires the user to upload a PDF or doc file to the server. Like Udemy, the HTTP request for Smallpdf to apply for the upload credentials to Amazon S3 is sent by the client. Unlike Udemy, the HTTP request sent by the client only includes the storage location and the file name in the Bucket. The rest of the field in the upload credential adopts the configured by the Smallpdf administrator by default. This measure reduces the user's controllability of the uploaded credential information and improves security to a certain extent. Nonetheless, the upload credential dispatched to the user is valid for up to 9 hours, and the upload credential does not limit the file type and size. Consistent with the threat to Udemy, an attacker can use this flaw to overwrite any file in the Bucket. In our tests, we overwrite the $user_{Bob}$'s Docx file in Amazon S3 with a malicious Docx file generated by CVE-2017-11882 and CVE-2021-40444 [52], [53]. When $user_{Bob}$ downloaded the overwritten file to his personal computer and accessed it, we gained control of $user_{Bob}$'s personal computer.

V. Root Causes and Mitigation

In this section, we discuss the root causes of cloud storage service vulnerabilities and provide mitigation methods to defend against the vulnerabilities.

A. Root Cause Analysis

The web user directly uploads files to the cloud storage service. Although the upload process is simplified, the server load is reduced, and the user experience is improved. As aforementioned, the theoretical analysis and case study analysis highlight the introduction of new role and server configuration errors, and cloud storage service vulnerabilities exist in the interaction process between users, web servers, and cloud storage services. After detailed analysis, we identify the root causes of the attacks as follows.

1) Improper Security Design of the Cloud Storage Service: Cloud storage service providers are responsible for designing security policies for website developers and users. The complexity of the three-part interactions brings more attack surface, and any improper design or configuration may cause serious security threats. For instance, in the process of users directly uploading files to the cloud storage service, it is important to verify the legal identity of the user and whether the uploaded file complies with security requirements. However, our experimental results show that many cloud storage services do not provide satisfying security design.

2) Insecure Implementations of Website Developers: Since cloud storage services provide users with various interfaces, different interfaces have different configurations. When website developers configure cloud storage services, cloud storage service providers do not guide them to implement the best security practices. Developers configure related policies according to the usage requirements of web servers and rarely pay attention to the security threats they face. These configurations that lack security practices provide the basis for cloud storage service vulnerability attacks.

Some websites (e.g., Google, YouTube) limit the times the upload credential can be used when configuring the cloud storage service and do not allow the upload credential to be used multiple times. Conversely, some websites (e.g., Udemy, Academia) ignore this problem in configuring the cloud storage service so that the upload credential has an excessive validity period and can be used multiple times. The degree of developers' understanding of security practices in new scenarios largely affects the security of websites using cloud storage services.

B. Mitigation

The development of cloud storage technology must prioritize security. It is essential to discuss security measures in this scenario to mitigate vulnerabilities, protect user privacy, and ensure server billing security. In a security issue of this scenario, the web user can be the victim or the attacker. Therefore, we focus on the defense measures for web servers and cloud storage services that should be implemented to mitigate these issues.

1) Upload Credential Management: Web servers and cloud storage services should strengthen the management of upload credentials and mitigate resource abuse caused by users getting a large number of upload credentials. One practical mitigation strategy is that a user can only apply for one upload credential within the expiry date of the current upload credential and make the upload credential one-time only, which Google and Live have already deployed. For anonymous users, managing upload credentials is also necessary, web servers and cloud storage services can dispatch one upload credential per IP address within a specific timeframe.

2) Principle of Least Privilege: The upload credentials should follow the least privilege principle. Ordinary web users should be limited to only uploading files to designated buckets without the ability to list buckets or read files. For instance, for CSDN's file stealing vulnerability, they resolved the issue by strictly limiting the permissions associated with the upload credentials. Specifically, they removed the GetBucket and GetObject permissions, leaving only the permission to upload files. The file storage location, file size, and file type in the upload credential should be signed by default to prevent attackers from malicious tampering. For example, Udemy added a detection mechanism for client-uploaded file information on the server side to prevent malicious users from arbitrarily forging uploaded file information. Moreover, cloud storage services should set Bucket permissions to private by default. When the website developer sets the Bucket permission to `public-read`, the cloud storage service must remind users of possible security issues.

3) Callback Notification Verification: Cloud storage services should implement a callback notification interface and sign the callback notification information. The web server must prioritize signature verification of the received callback notifications to prevent attackers from forging or tampering with the notification content. Although the client spends more waiting time due to the callback request and response process added to the callback notification, it can significantly improve the security of web servers and cloud storage services. It can enable website developers to grasp the usage of cloud storage services better. For instance, CSDN switched to Huawei OBS and implemented strict signature verification for callback notifications to fix the callback notification spoofing vulnerability.

We will release our semi-automated security check tool for HTTP POST credentials and STS temporary upload credentials in Amazon S3 and Alibaba Cloud OSS publicly on GitHub[4]. This tool enables website developers to detect and discover possible vulnerabilities in the website's use of cloud storage services. By inputting the interactive packet containing the upload credential, the tool analyzes the upload credentials in the relevant format and assesses any vulnerabilities in the upload process.

VI. Disclosure and Response

We have promptly reported the discovered vulnerabilities to the security teams of each website and actively contacted them to help mitigate the detected issues. We reported some cloud storage service vulnerabilities to CNVD and got 7 CNVD IDs. The results of our contacts with various website security teams are summarized as follows.

A. Google

They attributed this vulnerability to a resource abuse vulnerability and thanked us for our work.

B. Reddit

We report vulnerabilities we find to them via HackerOne. They classified the issue we report to them as a DoS vulnerability and thanked us for our work.

C. CSDN

They actively communicated with us and confirmed the issues on the website. Regarding the file stealing vulnerability, CSDN implemented strict permission controls on upload credentials to ensure that they are only authorized for file uploads. For the callback notification spoofing vulnerability, they identified it as a flaw in the implementation of callback notifications in Alibaba Cloud OSS and they reported this vulnerability to Alibaba Cloud OSS. At present, CSDN has switched to HUAWEI OBS (Object Storage Service) [54].

D. Notion

After we report the security issues of the cloud storage service to their security team, they immediately responded to our report and actively communicated with us about the specific details of the cloud storage service vulnerability. Lastly, the Notion Labs, Inc. team has indicated they are already aware of this scenario.

E. Realtor

They confirmed the cloud storage service vulnerability we report to them. They said that while they do not currently have a paid bug bounty program, they appreciate our efforts to disclose the security vulnerabilities we find responsibly.

F. Smallpdf

We report the discovered cloud storage service vulnerability to their security team and provided a video of the exploit. They said they used a temporary storage solution to mitigate the impact of the vulnerability.

G. Udemy

We report vulnerabilities for the cloud storage services in Udemy to their security team via HackerOne. They actively contacted us to understand the specific triggering process of the vulnerability, and they confirmed that the above vulnerability exists in Udemy.

H. Fiverr

We report the cloud storage service vulnerability to their security team on the Fiverr website. They actively communicated with us and invited us to submit detailed bug reports to them on the Bugcrowd platform. They confirmed the above vulnerability that exists in Fiverr based on the report, identified the vulnerability in the report as a valid issue, and classified the issue as an unrestricted file upload.

[4]https://github.com/Cyc1e183/Cloud-Storage-Security-Check

I. Academia

We report the cloud storage service vulnerability to their security team via the email address on the Academia web page. They accepted our report and confirmed the vulnerability. At the same time, they provide a bonus of $100 for our efforts.

J. Others

We have sent vulnerability reports to other websites with the cloud storage services' security vulnerabilities and look forward to receiving their feedback. We have got 7 CNVD IDs.

VII. Related Work

A. Security of Cloud Service Credentials

The increasing use and adoption of cloud services have brought greater attention to the security issues surrounding user credentials used for authentication. The previous studies on cloud service credential security have exposed security issues arising from the credentials leaks in source code repositories [9], [55], [56] and mobile applications [8], [10]. These issues are mainly caused by developers not clearing private data in the source code due to carelessness before uploading it to the source code repository and incorrect use of cloud service credentials. Besides that, previous studies revealed that lack of authentication, misuse of user credentials (such as root user credentials and regular credentials) in authentication, residual cloud credentials, or misconfiguration of user permissions in the authorization are the root causes of the problem of mobile application cloud data leakage [7], [8], [57]. The study for the user credentials contemporaneous with ours has also revealed security issues such as data leaks. They focus on common user credentials (e.g., IAM users) with excessive privileges in mobile apps [11]. Unlike the studies mentioned above which the user credentials can be extracted from the mobile applications, the cloud service user credentials are stored on the website server in our research scenario. Our work focuses on the security issues caused by the design and misuse of file upload credentials in website scenarios, in which the credentials used for file upload are usually temporarily generated according to the settings of the website developer and the file information to be uploaded. As far as we know, we are the first to research this scenario. From this perspective, we analyze the defects in access control and permission settings in the file uploading process and propose mitigation measures to improve the data security of cloud storage.

B. Security of Cloud Service Architecture

Security issues in different areas of cloud services architecture have also attracted much attention. Ristenpart et al. [58] pointed out that the use of virtualization allows third-party cloud providers to maximize the utilization of their sunk capital costs by multiplexing many customer VMs across a shared physical infrastructure. They exploited shared physical computing resources in an IaaS environment to implement cross-VM side-channel attacks. Borgolte et al. [59] studied the security issues caused by the trust relationship between DNS records and IP. They find a substantial number of stale DNS records that point to available IP addresses in clouds that are still actively attempting to be accessed. An attacker can allocate IP addresses to which stale DNS records point and exploit residual trust in the domain name for phishing, receiving and sending emails, or possibly distributing code to clients that load remote code from the domain, etc. Pauley et al. [60] explored a broad class of attacks on clouds which they refer to as cloud squatting. They exposed the potential configurations in cloud services that allow attackers to use these potential configurations to obtain sensitive information of prior tenants after allocating resources (e.g., IP addresses) in cloud services. The data management issues have also received attention [61], [62], [63], [64]. The requirements of cloud storage for data security are mainly reflected in the following aspects: data confidentiality, data integrity, data availability, fine-grained access control, anti-leakage and secure data sharing within dynamic groups, etc [12], [13], [14], [15], [65].

VIII. Conclusion

Nowadays, more and more websites leverage cloud services for file storage. Directly uploading files from web users to cloud storage is one of the most mainstream scenarios. In this paper, we systematically analyze the security risks in this scenario. We conduct an in-depth analysis of the three critical stages in this scenario, including *upload credential requesting and dispatching*, *file uploading and verification*, and *callback notification and response*. Based on the analysis, we identify six new types of vulnerabilities in this scenario, including *unrestricted upload credential acquisition (V1)*, *upload credentials validity flaw (V2)*, *unrestricted file types and file size (V3)*, *file overwriting (V4)*, *file stealing (V5)*, and *callback notification spoofing (V6)*. These vulnerabilities may pose serious security threats to users, web services, and cloud storage services.

To investigate the real-world impacts caused by these vulnerabilities, we perform a measurement of the existing popular websites that use public cloud services. Surprisingly, all the tested websites have at least one vulnerability. We find 79 new vulnerabilities during the measurement, report them to the corresponding security teams or communities, and receive positive responses from some of them. We hope our findings can shed light on future research on the security of cloud storage services.

IX. Ethics Consideration

The main focus of this research is to examine the vulnerability verification process and the potential negative consequences of vulnerability disclosure. We created two accounts, $user_{Trudy}$ and $user_{Bob}$, on each website to simulate an attacker and a victim, respectively. The vulnerability verification was confined to these two accounts and does not pose any security threat to other users. For more severe negative impacts, such as DoS attacks, we did not conduct direct empirical testing on the target website. Instead, we verified the websites' security

teams about vulnerabilities and their negative impact. It is important to emphasize that we reported the vulnerability to the website's security team as soon as we discovered it, and the website has fixed the vulnerabilities that could cause harm to it.

ACKNOWLEDGMENT

The authors sincerely appreciate the reviewers for their valuable comments to improve their article.

REFERENCES

[1] P. Yang, N. Xiong, and J. Ren, "Data security and privacy protection for cloud storage: A survey," *IEEE Access*, vol. 8, pp. 131723–131740, 2020.

[2] (2021). *Gmail Statistics, Users, Growth and Facts for 2021*. [Online]. Available: https://saasscout.com/statistics/gmail-statistics/

[3] (2019). *Reddit's 2019 Year in Review*. [Online]. Available: https://www.redditinc.com/blog/reddits-2019-year-in-review/

[4] R. Nachiappan, B. Javadi, R. N. Calheiros, and K. M. Matawie, "Cloud storage reliability for big data applications: A state of the art survey," *J. Netw. Comput. Appl.*, vol. 97, pp. 35–47, Nov. 2017.

[5] T. Lee, S. Wi, S. Lee, and S. Son, "FUSE: Finding file upload bugs via penetration testing," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–11.

[6] Y. Chen, Y. Li, Z. Pan, Y. Lu, J. Chen, and S. Ji, "URadar: Discovering unrestricted file upload vulnerabilities via adaptive dynamic testing," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1251–1266, 2024.

[7] C. Zuo, Z. Lin, and Y. Zhang, "Why does your data leak? Uncovering the data leakage in cloud from mobile apps," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2019, pp. 1296–1310.

[8] Y. Zhou, L. Wu, Z. Wang, and X. Jiang, "Harvesting developer credentials in Android apps," in *Proc. 8th ACM Conf. Secur. Privacy Wireless Mobile Netw.*, Jun. 2015, pp. 1–12.

[9] M. Meli, M. R. McNiece, and B. Reaves, "How bad can it git? Characterizing secret leakage in public GitHub repositories," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2019, pp. 1–18.

[10] H. Wen, J. Li, Y. Zhang, and D. Gu, "An empirical study of SDK credential misuse in iOS apps," in *Proc. 25th Asia–Pacific Softw. Eng. Conf. (APSEC)*, Dec. 2018, pp. 258–267.

[11] X. Wang, Y. Sun, S. Nanda, and X. F. Wang, "Credit karma: Understanding security implications of exposed cloud services through automated capability inference," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur.)*, 2023, pp. 6007–6024.

[12] T. Bhatia and A. K. Verma, "Data security in mobile cloud computing paradigm: A survey, taxonomy and open research issues," *J. Supercomput.*, vol. 73, no. 6, pp. 2558–2631, Jun. 2017.

[13] Z. Xia, N. N. Xiong, A. V. Vasilakos, and X. Sun, "EPCBIR: An efficient and privacy-preserving content-based image retrieval scheme in cloud computing," *Inf. Sci.*, vol. 387, pp. 195–204, May 2017.

[14] D. He, N. Kumar, S. Zeadally, and H. Wang, "Certificateless provable data possession scheme for cloud-based smart grid data management systems," *IEEE Trans. Ind. Informat.*, vol. 14, no. 3, pp. 1232–1241, Mar. 2018.

[15] J. Shen, J. Shen, X. Chen, X. Huang, and W. Susilo, "An efficient public auditing protocol with novel dynamic structure for cloud data," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 10, pp. 2402–2415, Oct. 2017.

[16] J. Cable, D. Gregory, L. Izhikevich, and Z. Durumeric, "Stratosphere: Finding vulnerable cloud storage buckets," in *Proc. 24th Int. Symp. Res. Attacks, Intrusions Defenses*, Oct. 2021, pp. 399–411.

[17] *Academia.edu*. Accessed: 2022. [Online]. Available: https://www.academia.edu/

[18] *Uploading to Amazon S3 Directly From a Web or Mobile Application*. Accessed: 2022. [Online]. Available: https://aws.amazon.com/blogs/compute/uploading-to-amazon-s3-directly-from-a-web-or-mobile-application/

[19] *Uploading Objects to OSS Directly From Clients*. Accessed: 2022. [Online]. Available: https://help.aliyun.com/zh/oss/use-cases/uploading-objects-to-oss-directly-from-clients/

[20] *Comparing Aws Account Root User Credentials and IAM User Credentials*. Accessed: 2022. [Online]. Available: https://docs.aws.amazon.com/accounts/latest/reference/root-user-vs-iam.html

[21] *Introduction to Resource Access Management—Alibaba Cloud Document Center*. Accessed: 2022. [Online]. Available: https://www.alibabacloud.com/help/en/resource-access-management

[22] *Use STS Temporary Access Credentials to Access OSS*. Accessed: 2022. [Online]. Available: https://help.aliyun.com/document_detail/100624.html

[23] Y. Tang, P. P. C. Lee, J. C. S. Lui, and R. Perlman, "Secure overlay cloud storage with access control and assured deletion," *IEEE Trans. Dependable Secure Comput.*, vol. 9, no. 6, pp. 903–916, Nov. 2012.

[24] Y. Tang, P. P. C. Lee, J. C. S. Lui, and R. Perlman, "FADE: Secure overlay cloud storage with file assured deletion," in *Proc. Int. Conf. Secur. Privacy Commun. Syst.* Cham, Switzerland: Springer, Jan. 2010, pp. 380–397.

[25] M. Chatterjee, P. Datta, F. Abri, A. Siami Namin, and K. S. Jones, "Abuse of the cloud as an attack platform," in *Proc. IEEE 44th Annu. Comput., Softw., Appl. Conf. (COMPSAC)*, Jul. 2020, pp. 1091–1092.

[26] (2023). *Amazon S3 Simple Storage Service Pricing—Amazon Web Services*. [Online]. Available: https://aws.amazon.com/s3/pricing/?nc1=h_ls

[27] (2023). *Azure Blob Storage Pricing — Microsoft Azure*. [Online]. Available: https://azure.microsoft.com/en-us/pricing/details/storage/blobs/

[28] (2023). *OSS (Object Storage Service) Pricing & Purchasing Methods—Alibaba Cloud*. [Online]. Available: https://www.alibabacloud.com/product/oss/pricing

[29] M. Wachs, L. Xu, A. Kanevsky, and G. R. Ganger, "Exertion-based billing for cloud storage access," in *Proc. 3rd USENIX Workshop Hot Topics Cloud Comput. (HotCloud 11)*, Jun. 2011, p. 7.

[30] G. Hong, M. Wu, P. Chen, X. Liao, G. Ye, and M. Yang, "Understanding and detecting abused image hosting modules as malicious services," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2023, pp. 3213–3227.

[31] Y. Lv, W. Shi, W. Zhang, H. Lu, and Z. Tian, "Don't trust the clouds easily: The insecurity of content security policy based on object storage," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10462–10470, Jun. 2023.

[32] L. Weichselbaum, M. Spagnuolo, S. Lekies, and A. Janc, "CSP is dead, long live CSP! On the insecurity of whitelists and the future of content security policy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2016, pp. 1376–1387.

[33] S. Calzavara, A. Rabitti, and M. Bugliesi, "Content security problems: Evaluating the effectiveness of content security policy in the wild," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2016, pp. 1365–1375.

[34] G. E. Rodríguez, J. G. Torres, P. Flores, and D. E. Benavides, "Cross-site scripting (XSS) attacks and mitigation: A survey," *Comput. Netw.*, vol. 166, Jan. 2020, Art. no. 106960.

[35] G. Pellegrino, M. Johns, S. Koch, M. Backes, and C. Rossow, "Deemon: Detecting CSRF with dynamic analysis and property graphs," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1757–1771.

[36] D. Kim and H. Kim, "Performing clickjacking attacks in the wild: 99% are still vulnerable!," in *Proc. 1st Int. Conf. Softw. Secur. Assurance (ICSSA)*, Jul. 2015, pp. 25–29.

[37] D. Bates, A. Barth, and C. Jackson, "Regular expressions considered harmful in client-side XSS filters," in *Proc. 19th Int. Conf. World Wide Web*, Apr. 2010, pp. 91–100.

[38] M. Mohammadi, B. Chu, H. R. Lipford, and E. Murphy-Hill, "Automatic Web security unit testing: XSS vulnerability detection," in *Proc. IEEE/ACM 11th Int. Workshop Autom. Softw. Test (AST)*, May 2016, pp. 78–84.

[39] Amazon.(2022). *Cloud Object Storage—amazon S3*. [Online]. Available: https://aws.amazon.com/s3/

[40] *Cloud Computing Services — Microsoft Azure*. Accessed: 2022. [Online]. Available: https://azure.microsoft.com/

[41] *Object Storage Data Processing-Cos Data Processing-Data Processing Solution-Tencent Cloud*. Accessed: 2022. [Online]. Available: https://cloud.tencent.com/product/cos

[42] *Object Storage BOS*. Accessed: 2022. [Online]. Available: https://cloud.baidu.com/product/bos.html

[43] *About Object Operations*. Accessed: 2022. [Online]. Available: https://help.aliyun.com/document_detail/31977.html

[44] *Amazon S3 Rest API Introduction—Amazon Simple Storage Service*. Accessed: 2022. [Online]. Available: https://docs.aws.amazon.com/zh_cn/AmazonS3/latest/API/Welcome.html
[45] J. Qian, S. Hinrichs, and K. Nahrstedt, "ACLA: A framework for access control list (ACL) analysis and optimization," in *Communications and Multimedia Security Issues of the New Century: IFIP TC6 / TC11 Fifth Joint Working Conference on Communications and Multimedia Security (CMS'01) May 21–22, 2001, Darmstadt, Germany*. Cham, Switzerland: Springer, 2001, pp. 197–211.
[46] *GitHub*. Accessed: 2022. [Online]. Available: https://github.com/qingzhenyun/qingzhenyun-api-gateway/blob/5cd0eb45e82d6276b400aee6dc871e3-a07bb4e03/src/app/web/controllers/v1/store.js\#L64
[47] E. Trickel et al., "Toss a fault to your witcher: Applying grey-box coverage-guided mutational fuzzing to detect SQL and command injection vulnerabilities," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2023, pp. 2658–2675.
[48] *Post Policy—Amazon Simple Storage Service*. Accessed: 2022. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/API/sigv4-HTTPPOSTConstructPolicy.html
[49] *Response Header*. Accessed: 2022. [Online]. Available: https://developer.mozilla.org/zh-CN/docs/Glossary/Response_header
[50] *Authenticating Requests: Browser-Based Uploads Using Post (AWS Signature Version 4)*. Accessed: 2022. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/API/sigv4-authentication-HTTPPOST.html
[51] *Authenticating Requests: Using Query Parameters (AWS Signature Version 4)*. Accessed: 2022. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/API/sigv4-query-string-auth.html
[52] *CVE-2017-11882*. Accessed: 2022. [Online]. Available: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-11882
[53] *CVE-2021-40444*. Accessed: 2022. [Online]. Available: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-40444
[54] *Object Storage Service (OBS)—Huawei Cloud*. Accessed: 2022. [Online]. Available: https://www.huaweicloud.com/intl/en-us/product/obs.html
[55] V. S. Sinha, D. Saha, P. Dhoolia, R. Padhye, and S. Mani, "Detecting and mitigating secret-key leaks in source code repositories," in *Proc. IEEE/ACM 12th Work. Conf. Mining Softw. Repositories*, May 2015, pp. 396–400.
[56] Z. Yu Ding, B. Khakshoor, J. Paglierani, and M. Rajpal, "Sniffing for codebase secret leaks with known production secrets in industry," 2020, *arXiv:2008.05997*.
[57] A. D. Diego, "Automatic extraction of API keys from Android applications," Ph.D. dissertation, Dept. Eng., Università degli Studi di Roma Tor Vergata, Rome, Italy, 2017.
[58] T. Ristenpart, E. Tromer, H. Shacham, and S. Savage, "Hey, you, get off of my cloud: Exploring information leakage in third-party compute clouds," in *Proc. 16th ACM Conf. Comput. Commun. Secur.*, Nov. 2009, pp. 199–212.
[59] K. Borgolte, T. Fiebig, S. Hao, C. Kruegel, and G. Vigna, "Cloud strife: Mitigating the security risks of domain-validated certificates," in *Proc. Appl. Netw. Res. Workshop (ANRW)*. New York, NY, USA: ACM, 2018, p. 4, doi: 10.1145/3232755.3232859. [Online]. Available: https://dl.acm.org/doi/10.1145/3232755.3232859
[60] E. Pauley, R. Sheatsley, B. Hoak, Q. Burke, Y. Beugin, and P. McDaniel, "Measuring and mitigating the risk of IP reuse on public clouds," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2022, pp. 558–575.
[61] X. Zhang, H.-T. Du, J.-Q. Chen, Y. Lin, and L.-J. Zeng, "Ensure data security in cloud storage," in *Proc. Int. Conf. Netw. Comput. Inf. Secur.*, vol. 1, May 2011, pp. 284–287.
[62] A. Markandey, P. Dhamdhere, and Y. Gajmal, "Data access security in cloud computing: A review," in *Proc. Int. Conf. Comput., Power Commun. Technol. (GUCON)*, Sep. 2018, pp. 633–636.
[63] D. Zhe, W. Qinghong, S. Naizheng, and Z. Yuhan, "Study on data security policy based on cloud storage," in *Proc. IEEE 3rd Int. Conf. Big Data Secur. Cloud (Bigdatasecurity) Int. Conf. High Perform. Smart Comput. (HPSC), IEEE Int. Conf. Intell. Data Secur. (IDS)*, May 2017, pp. 145–149.
[64] Y. Zhang, C. Xu, and X. S. Shen, *Data Security in Cloud Storage*. Cham, Switzerland: Springer, 2020.
[65] Z. Xia, T. Shi, N. N. Xiong, X. Sun, and B. Jeon, "A privacy-preserving handwritten signature verification method using combinational features and secure KNN," *IEEE Access*, vol. 6, pp. 46695–46705, 2018.

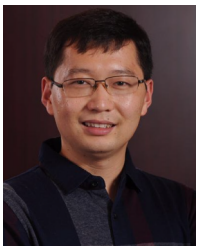
Yuanchao Chen received the bachelor's degree in network engineering from the National University of Defense Technology, Hefei, China, in 2019, where he is currently pursuing the Ph.D. degree. His research interests include web application security, vulnerability discovery, and cloud security.

Yuwei Li received the Ph.D. degree in computer science and technology from Zhejiang University, Hangzhou, China, in 2021. She is currently an Associate Professor with the College of Electronic Engineering, National University of Defense Technology. Her interested research interests include system and software security, fuzz testing, and vulnerability detection.

Yuliang Lu was born in China in 1964. He received the B.Sc. (Hons.) and M.Sc. degrees in computer application from Southeast University, China, in 1985 and 1988, respectively. He is currently a Professor with the National University of Defense Technology, Hefei, China. His research interests include computer application and information processing.

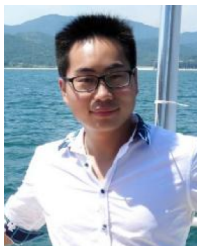
Zulie Pan received the Ph.D. degree from the Electronic Engineering Institute, Hefei, China. He is currently a Professor with the College of Electronic Engineering, National University of Defense Technology. His research interests include vulnerability discovery, network security, and computer science.

Yuan Chen received the bachelor's degree in biology from Zhejiang University, where he is currently pursuing the Ph.D. degree in computer science and technology. His current research interests include web security and system security.



Shouling Ji (Member, IEEE) received the Ph.D. degree in electrical and computer engineering from Georgia Institute of Technology and the Ph.D. degree in computer science from Georgia State University. He is currently a ZJU 100-Young Professor with the College of Computer Science and Technology, Zhejiang University, and a Research Faculty with the School of Electrical and Computer Engineering, Georgia Institute of Technology. His current research interests include AI security, data-driven security, privacy, and data analytics. He is a member of ACM. He was the Membership Chair of the IEEE Student Branch at Georgia State from 2012 to 2013.

Yu Chen received the master's degree from the College of Electronic Engineering, National University of Defense Technology, Anhui, China, where he is currently pursuing the Ph.D. degree. His research interests include web application security and web vulnerability discovery.



Yang Li received the master's degree from the Electronic Engineering Institute, Hefei, China. He is currently an Associate Professor with the College of Electronic Engineering, National University of Defense Technology, Anhui, China. His research interests include vulnerability discovery, network security, and computer science.

Yi Shen is currently an Associate Professor with the College of Electronic Engineering, National University of Defense Technology, Hefei, China. His research interests include web application security and program testing.